

PC Bottleneck Predictor for Gaming Systems

Project Category: Machine Learning & Systems Performance

Name: Bandaru Venkat Pavan

Role: Project Lead

24cs3015@rgitp.ac.in

Name: Gaddam Pranav

Role: Team Member

24cs3019@rgipt.ac.in

Name: Gutta Kaushik

Role: Team Member

24ms3019@rgipt.ac.in

1 Introduction

Gaming PC performance is a topic of significant interest to both casual and competitive gamers. A common and often misunderstood problem is the concept of a hardware *bottleneck* — a condition in which one component in a system limits the performance of another. Specifically, if a GPU is significantly more powerful than the paired CPU, the CPU cannot supply frames fast enough to keep the GPU fully utilised. Conversely, a weak GPU paired with a powerful CPU leaves the processor idle for most of each frame. Both conditions result in suboptimal performance and unnecessary hardware cost.

This project builds a machine learning system that predicts the bottleneck percentage for any given CPU and GPU pairing. The input is a pair of hardware components specified by name; the output is a signed bottleneck score in the range $[-100, +100]$, where a positive score indicates a GPU bottleneck, a negative score indicates a CPU bottleneck, and a score near zero indicates a well-matched, balanced system.

We use real benchmark data from PassMark — a widely used hardware benchmarking platform — as the foundation for our dataset. We engineer a synthetic target variable from CPU and GPU benchmark scores and train two regression models entirely from scratch using only NumPy, without any scikit-learn or other ML library. Our best model achieves an R^2 of 0.73 on held-out test data with a cross-validated standard deviation of ± 0.019 , confirming stable generalisation.

2 Related Work

Hardware performance prediction and bottleneck analysis have historically been addressed through simulation-based methods and rule-of-thumb heuristics rather than data-driven machine learning. Popular tools such as PC-Builds.com and Bottleneck Calculator use fixed lookup tables and manually crafted ratio thresholds between benchmark scores. These approaches lack the ability to generalise to new hardware and do not account for interaction effects between CPU core count, thermal design power (TDP), and GPU compute performance.

Regression-based approaches to hardware performance

modelling have been explored in the systems literature. Work by Ipek et al. demonstrated that artificial neural networks can predict memory system performance from hardware event counters [1]. More recent work on roofline modelling characterises compute versus memory bandwidth bottlenecks in HPC workloads [2]. Our work differs in that we focus on consumer gaming hardware, use publicly available benchmark scores as features, and derive a novel continuous bottleneck target from the ratio of normalised benchmark scores rather than relying on binary classification.

To our knowledge, no prior published work applies supervised regression with polynomial feature engineering and L2 regularisation to the specific problem of CPU-GPU bottleneck prediction for consumer gaming systems. This project fills that gap using real benchmark data from 1,114 desktop CPUs and 289 desktop GPUs.

3 Dataset and Features

3.1 Dataset and Split

We use two publicly available benchmark datasets from PassMark Performance Test [3, 4] — one for CPUs and one for GPUs. After filtering to Desktop-category components and dropping rows with missing values, we retain 1,114 CPUs and 289 GPUs. We construct our training universe by taking the Cartesian product of all CPU-GPU pairs, yielding 321,946 unique build configurations. This cross-join approach allows us to train on every possible pairing without requiring paired real-world measurements.

The dataset is split into a training set of 257,556 samples (80%) and a test set of 64,390 samples (20%) using a fixed random seed of 42 for reproducibility. For cross-validation, we draw a random sample of 20,000 rows and apply 5-fold CV to confirm that reported metrics are not an artefact of a single lucky split.

3.2 Target Engineering

Neither dataset contains a ground-truth bottleneck label. We engineer the target variable from the benchmark scores using the following formula:

$$\text{bottleneck}_{\%} = 100 \times \frac{\text{gpu}_{\text{norm}} - \text{cpu}_{\text{norm}}}{\text{gpu}_{\text{norm}} + \text{cpu}_{\text{norm}} + \varepsilon} \quad (1)$$

where cpu_{norm} and gpu_{norm} are the min-max normalised benchmark scores (cpuMark and G3Dmark respectively), and $\varepsilon = 10^{-9}$ prevents division by zero. The resulting target lies in $[-100, +100]$. A value of $+100$ indicates the GPU completely overwhelms the CPU; -100 indicates the reverse; 0 indicates perfect balance.

3.3 Features and Pre-processing

We construct four base features from each CPU-GPU pair (Table 1). These features are subsequently expanded to 14 features using degree-2 polynomial transformation, adding all squared and cross-product terms. The full 14-feature matrix is then Z-score standardised (mean subtracted, divided by standard deviation) so that gradient descent converges at a uniform rate across all features.

Table 1: Base feature set. Each pair of 4 base features is expanded to 14 via degree-2 polynomial transformation.

Feature	Source	Description
cpuMark	PassMark CPU	Multi-threaded benchmark score
G3Dmark	PassMark GPU	3D graphics score (gaming proxy)
ratio	Engineered	$\text{gpu}_{\text{norm}} / (\text{cpu}_{\text{norm}} + \varepsilon)$
cores	PassMark CPU	Physical CPU core count

4 Methods

All models are implemented entirely from scratch using NumPy — no scikit-learn, no PyTorch, no TensorFlow. The project follows strict OOP principles with 15 separate Python files, each containing a single class with a single responsibility.

4.1 Linear Regression with Gradient Descent

Our baseline model is multivariate linear regression. The model predicts:

$$\hat{y} = \mathbf{X}\mathbf{w} + b \quad (2)$$

where $\mathbf{X} \in \mathbb{R}^{m \times n}$ is the design matrix, $\mathbf{w} \in \mathbb{R}^n$ is the weight vector, and b is the scalar bias. Training minimises the mean squared error cost:

$$J(\mathbf{w}, b) = \frac{1}{2m} \|\mathbf{X}\mathbf{w} + b - \mathbf{y}\|^2 \quad (3)$$

via batch gradient descent. The vectorised update rules at each iteration are:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{m} \mathbf{X}^T (\mathbf{X}\mathbf{w} + b - \mathbf{y}) \quad (4)$$

$$b \leftarrow b - \frac{\alpha}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \quad (5)$$

where α is the learning rate and weights are initialised to zero. The factor of $\frac{1}{2}$ in the cost is a mathematical convenience — it cancels with the exponent when differentiating the squared term, producing a cleaner gradient expression.

4.2 Ridge Regression (L2 Regularisation)

When the feature space is expanded from 4 to 14 dimensions via polynomial transformation, the model gains the capacity to overfit. Ridge Regression prevents this by adding a penalty term to the cost:

$$J_{\text{ridge}}(\mathbf{w}, b) = \frac{1}{2m} \|\mathbf{X}\mathbf{w} + b - \mathbf{y}\|^2 + \frac{\lambda}{2m} \|\mathbf{w}\|^2 \quad (6)$$

The gradient with respect to $\mathbf{w}$ gains an additional regularisation term:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{m} \mathbf{X}^T (\mathbf{X}\mathbf{w} + b - \mathbf{y}) + \frac{\lambda}{m} \mathbf{w} \quad (7)$$

The bias b is intentionally excluded from the penalty — regularising the bias would shift all predictions uniformly without reducing model complexity. The hyperparameter λ controls the strength of regularisation: $\lambda = 0$ recovers plain linear regression; larger λ shrinks weights toward zero.

4.3 Polynomial Feature Engineering

Bottleneck percentage has a non-linear relationship with raw benchmark scores. To allow a linear model to fit this curve, we apply a degree-2 polynomial expansion to the base feature vector $\mathbf{x} = [x_1, x_2, x_3, x_4]$. This adds all squared terms x_i^2 and all cross-product terms $x_i x_j$ ($i \leq j$), expanding the feature count from 4 to 14. The model remains linear in the weights — it is the input representation that gains expressivity, not the model class.

Double standardisation. Polynomial terms such as x_i^2 have different scales than linear terms even after initial standardisation. We therefore apply Z-score normalisation twice: once to the 4-dimensional base feature matrix (before expansion), and once to the full 14-dimensional polynomial matrix. This prevents numerical overflow and ensures uniform gradient magnitude across all 14 features.

4.4 K-Fold Cross Validation

A single train/test split can be lucky or unlucky depending on which rows end up in each partition. We implement 5-fold cross-validation from scratch to produce a reliable estimate of generalisation performance. In each round, 4 folds are used for training and 1 is held out for evaluation. Every sample is evaluated exactly once. The reported metric is the mean R^2 and its standard deviation across all 5 folds.

5 Experiments

All models are trained with learning rate $\alpha = 0.1$ and 5,000 gradient descent iterations. Model comparison is performed on a 30,000-row random sample of the training data for computational efficiency; the winning model is then retrained on the full 257,556-row training set before final evaluation. The primary metric is R^2 on the held-out test set. We also report RMSE, MAE, and the overfit gap (train R^2 – test R^2).

We compare five model configurations — one Linear Regression and four Ridge Regression models with $\lambda \in \{0.01, 0.1, 1.0, 10.0\}$. All five operate on the same 14-feature polynomial-expanded, double-standardised input matrix.

Table 2: Model comparison on the test set (64,390 samples). ★ denotes the winning model retrained on the full training set. Overfit gap = train R^2 – test R^2 .

Model	Train R^2	Test R^2	RMSE	MAE	Gap
Linear Reg. ★	0.7322	0.7321	34.59	27.55	0.0001
Ridge $\lambda=0.01$	0.7322	0.7321	34.59	27.55	0.0001
Ridge $\lambda=0.1$	0.7322	0.7321	34.59	27.55	0.0001
Ridge $\lambda=1.0$	0.7322	0.7321	34.59	27.55	0.0001
Ridge $\lambda=10.0$	0.7320	0.7319	34.60	27.55	0.0001

The key finding is that all five models converge to the same test R^2 of 0.73 and the overfit gap is essentially zero across all λ values. This indicates that degree-2 polynomial features on this dataset are not prone to overfitting at this feature complexity. Linear Regression is selected as the winner on grounds of parsimony.

5.1 Cross-Validation Results

Table 3: 5-fold cross-validation results for the winning Linear Regression model (20,000-row sample).

Fold	R^2	RMSE
Fold 1	0.7467	33.91
Fold 2	0.7276	34.76
Fold 3	0.7306	34.91
Fold 4	0.6897	36.90
Fold 5	0.7340	34.25
Mean $\pm$ Std	0.7257 $\pm$ 0.019	34.95

The low standard deviation of ± 0.019 confirms model stability. The mean cross-validated R^2 of 0.7257 is consistent with the single-split test result of 0.7321, confirming no data-split optimism bias.

5.2 Sample Predictions

Table 4: Representative predictions from the final trained model.

CPU	GPU	Score	Verdict	Severity
Ryzen 5 5600X	RTX 3060 Ti	+50.93	GPU Bottleneck	Moderate
Ryzen 9 5900X	RX 6600	+43.32	GPU Bottleneck	Moderate
Ryzen 9 5900X	RTX 3080 Ti	−9.82	Balanced	Balanced

6 Discussion

The ceiling R^2 of 0.73 for polynomial linear models on this dataset is interpretable and expected. The bottleneck relationship is fundamentally governed by the ratio of normalised benchmark scores, which is a non-linear function that polynomial regression approximates but cannot perfectly represent. The remaining 27% of unexplained variance corresponds to architecture-specific effects — two CPUs with the same PassMark score may behave very differently in gaming due to differences in instruction-level parallelism, cache hierarchy, or game engine threading behaviour — none of which are captured by aggregate benchmark scores.

A further contributing factor is the synthetic nature of the target variable. While Equation (1) is theoretically motivated, it is not a directly measured ground truth. A more accurate target would require actual frame-time measurements across multiple game titles and resolutions for each CPU-GPU pair.

Optimizer analysis. We found that $\alpha = 0.1$ produced fast, stable convergence with 5,000 iterations. Gradient overflow (NaN values) occurred when polynomial features were not re-standardised after expansion — resolved by applying a second round of Z-score normalisation to the expanded 14-feature matrix.

Regularisation. The near-zero overfit gap across all λ values confirms that degree-2 polynomial features on standardised inputs do not induce overfitting on this 321k sample dataset. Ridge is included to demonstrate the concept and the gradient derivation, even though λ does not materially improve test performance here.

7 Conclusion and Future Work

We have demonstrated that a machine learning regression system can predict CPU-GPU bottleneck percentage from publicly available benchmark data with an R^2 of 0.73, entirely implemented from scratch without any ML libraries. Key contributions include: a novel bottleneck target engineering formula derived from normalised benchmark scores; a two-stage standardisation pipeline that prevents overflow in polynomial feature spaces; and a full OOP

project architecture with 15 single-responsibility classes including a model comparison engine and a 5-fold cross-validator.

Future work could explore several directions. First, replacing the synthetic target with real frame-time measurements from gaming benchmarks would produce a more accurate ground truth and likely improve model performance significantly. Second, a gradient-boosted decision tree (e.g. XGBoost) or a shallow neural network should be capable of capturing the non-linear component not accessible to polynomial regression, potentially pushing R^2 above 0.90. Third, the feature set could be enriched with game-specific parameters such as target resolution (1080p / 1440p / 4K) and game engine type, segmenting predictions into game-specific models. Finally, deploying the trained model as a REST API and connecting it to a frontend would make the tool accessible to the broader gaming community.

8 Contributions

Bandaru Venkat Pavan (Project Lead): Overall project architecture, OOP design, gradient descent implementation (LinearRegression, RidgeRegression), polynomial feature engineering, double-standardisation pipeline, model comparison framework, cross-validation, report writing, system integration.

Gaddam Pranav (Team Member): Data loading and preprocessing pipeline (DataLoader, Preprocessor), target variable engineering, cross-join dataset construction, model saving and loading (ModelSaver), lookup table design, prediction interface (Predictor).

Gutta Kaushik (Team Member): Evaluation metrics from scratch (Evaluator), visualisation suite (Visualiser — cost curves, predicted vs. actual, residual analysis, feature importance, model comparison bar chart), report generation (ReportGenerator), hyperparameter grid search, experiment logging.

References

- [1] E. Ipek, S. A. McKee, R. Caruana, B. R. de Supinski, and M. Schulz. Efficiently exploring architectural design spaces via predictive modeling. In *ASPLOS*, pp. 195–206, 2006.
- [2] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [3] PassMark Software. PassMark CPU Benchmark Charts. <https://www.cpubenchmark.net>, 2024.
- [4] PassMark Software. PassMark GPU Benchmark Charts. <https://www.videocardbenchmark.net>, 2024.
- [5] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- [6] A. E. Hoerl and R. W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.